In light of this, we examine GPT-2's task performance by using causal techniques to identify a **circuit**: a minimal computational subgraph of our model that suffices to compute the task [35, 14]. While many causal methods aim to identify important components (nodes) of models [48, 11, 17], the circuits methodology instead considers important *edges*. Circuits thus examine not only important nodes, but also their interactions, and how they work together to support model behavior.

Below, we explain the *path patching* technique, and how to use it to find circuits (Section 3.1). We then find a circuit for greater-than, and prove its correctness (Section 3.2). Finally, we assign semantics to the nodes and edges of our circuit (Section 3.3). All of our experiments use the `rust-circuit` library, and our code is available at `https://github.com/hannamw/gpt2-greater-than`. For information on how to apply this methodology to other problems, see Appendix H.

3.1 Path Patching

To find a circuit, we use path patching, introduced by Wang et al. [50] and further described by Goldowsky-Dill et al. [21]. This technique determines how important a model component (e.g. an attention head or MLP) is to a task, by altering that component's inputs and observing model behavior post-alteration. It is much like causal mediation analysis or interchange interventions [48, 17]; however, unlike these, it allows us to constrain our intervention's effects to a specific path.

To illustrate this, consider a model's forward pass on its inputs as a directed acyclical graph. Its nodes are components such as attention heads or MLPs. The input of a node v is the sum of the outputs of all nodes with a direct edge to v. GPT-2 can be thought of as such a graph flowing from its input tokens to its logits (and thereafter, its probabilities), as depicted in Figure 3.

In path patching, we specify new input tokens, and a path of components through which they will reach the logits. For example, if we want to ascertain the effects of MLP 10 on the logits, we can patch the direct path (MLP 10, logits) with new input, which we call the `01`-input: "The war lasted from the year 1701 to the year 17". We thus alter MLP 10's direct effects on the logits without changing its output to the attention and MLP of layer 11 (Figure 3). If the model's behavior (as indicated by its logits) changes, we can be sure that this is because MLP 10 is important to that behavior; it is not due to downstream components. Earlier methods like interchange interventions lack this distinction—when they alter a component, they affect all components downstream from it.

The specificity of path patching allows us to test detailed hypotheses. For example, imagine that we know that MLP 10 affects the logits both directly and via its effects on MLP 11. We want to know how important layer 10's attention is to the circuit via MLP 10. We can test this by patching two paths at once: (Attn 10, MLP 10, logits) and (Attn 10, MLP 10, MLP 11, logits), as in Figure 3. This allows us to pinpoint the relationship between precisely these two components, Attn 10 and MLP 10. This technique underpins our circuits approach: we search for a path starting in the inputs and ending in the logits that explains how our model performs the greater-than task.

To perform path patching, we need a new dataset that replaces a node's original inputs. To this end, we create the "`01`-dataset": we take each example in the original dataset and replace the last two digits `YY` of the start year with "01". If a component normally boosts logits of years>`YY`, patching it with the `01`-dataset will cause it to boost the logits of years > 01, inducing a larger error in the model.

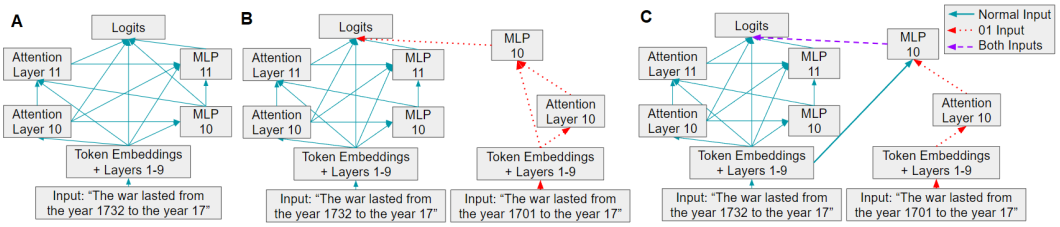

Figure 3: **A**: The computational graph of GPT-2, run on our normal dataset. **B**: GPT-2, where the (MLP 10, logits) path is patched to receive `01`-input. **C**: GPT-2, where the (Attn 10, MLP 10, logits) path receives `01`-input. Nodes receiving normal input have blue output; nodes receiving `01`-input have red output; nodes receiving both have purple output. Note that in B and C, there is no longer an edge connecting MLP 10 and the logits.

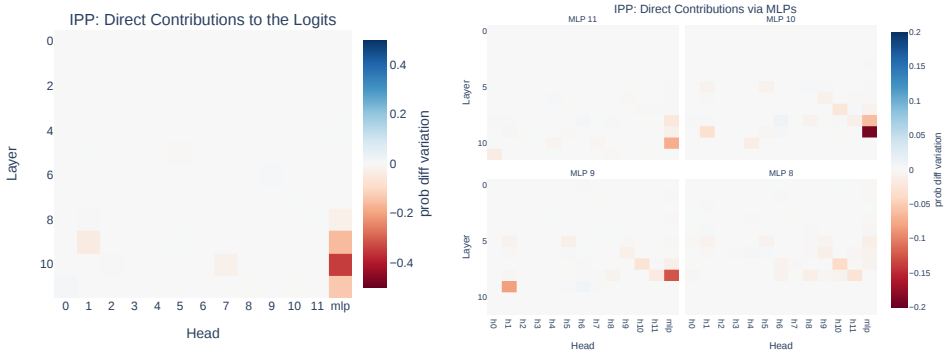

Figure 4: Iterative path-patching (IPP) heatmaps. Y-axis: layer of the component. X-axis: attention head number, or MLP. (X,Y): Change in probability difference induced by patching the corresponding component. **Left**: Heatmap for the path ((X,Y), logits). **Right**: Heatmaps for MLPs 8-11.

3.2 Circuit Components

MLPs We search for a circuit by identifying components that perform year-span prediction via their direct connection to the logits. We consider as potential components GPT-2's 144 attention heads (12 heads/layer$\times$12 layers), and 12 MLPs (1 per layer). We do so because the residual stream [14] that serves as input to the logits is simply the sum of these components' direct contributions (along with the token embeddings; we ignore these as they contain no `YY` information). If we consider each of these, we will not miss any components that contribute to this task. For details, see Appendix C.

We iteratively path patch each component's direct contributions to the logits, replacing its inputs with the `01`-dataset. In our earlier notation, for a component of interest C, we patch the path (C, logits), as in Figure 3 B, where C = MLP 10. We patch only one component at a time, and only at the end of the sentence; at other positions, these components cannot affect the logits directly.

After we patch a component, we run the model and record the probability difference, comparing it to that of the unpatched model. If patching a component caused model performance to change significantly, that component contributed to the model's computation of year-span prediction.

Figure 4 shows the results of this experiment; for computational reasons we run it using a smaller dataset (490 datapoints, 5 per year `YY`). The heatmap indicates that MLPs 8-11 are the most important direct contributors to the logits, along with a9.h1: attention layer 9's head 1. However, the MLPs cannot act alone: to compute years>`YY`, these MLPs at the end of the sentence must know the value of `YY`. But unlike attention heads, MLPs cannot attend to earlier tokens such as the `YY` token. Thus, we search for nodes that contribute to the circuit via these MLPs.

Attention Heads We find components that contribute to the circuit via the MLPs using more path patching. We start by patching components through MLP 11, since it is the furthest downstream; for a component of interest C, we patch (C, MLP 11, logits). We find that MLP 11 relies mostly on the 3 MLPs upstream of it (Figure 4), so we search for components that act via those MLPs.

We next find components that contribute to the circuit through MLP 10. For a given C, we patch (C, MLP 10, logits) and (C, MLP 10, MLP 11, logits), as in Figure 3 C. We do so because MLP 10 contributes directly to two nodes in our circuit, the logits and MLP 11, and we want to know which nodes contribute via MLP 10 to the entire circuit. We repeat this procedure for MLPs 9 and 8.

The results in Figure 4 indicate that MLPs rely heavily on other MLPs upstream of them. MLPs 8 and 9, the furthest upstream of our MLPs, also rely on attention heads. MLP 9 relies on a9.h1, while MLP 8 relies on a8.h11, a8.h8, a7.h10, a6.h9, a5.h5, and a5.h1; we add these to our circuit. Many of these attention heads can be seen to contribute to the logits directly, though more weakly than the MLPs do. For this reason, we also add these heads' direct connections to the logits to our circuit.

Figure 5 visualizes the circuit we have found. We could further develop this by specifying a circuit from the token inputs to the logits; indeed, we do so in Appendix B. However, the present circuit

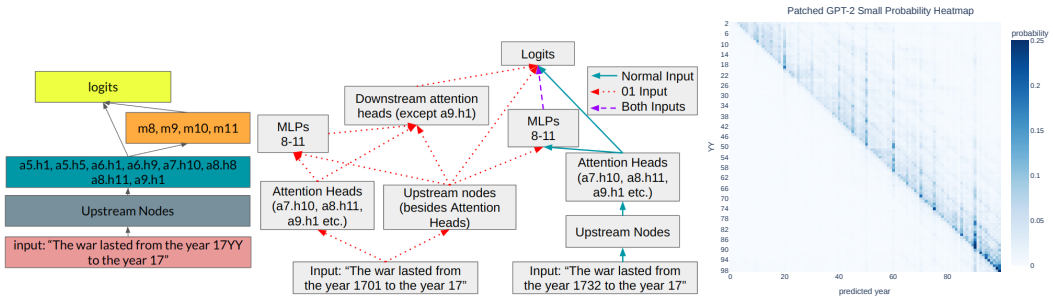

Figure 5: **Left**: Diagram of the year-span prediction circuit. **Center**: Diagram showing which GPT-2 components receive our standard dataset vs. our 01-dataset in the circuit evaluation experiment. **Right**: The probability heatmap (as in Figure 2) for the patched model.

already captures the most interesting portion of the model: the MLPs that compute greater-than. So, we instead provide evidence that our circuit is correct, and then analyze its constituent parts.

Evaluation Having defined our circuit, we perform another path-patching experiment to ensure it is correct. In this experiment, we give most of the model inputs from the 01-dataset. The model only receives our standard dataset via the paths specified in our circuit. So, our attention heads' contributions to the logits are backed by the standard dataset, as are their contributions to the MLPs, and the MLPs' contributions to one another. But, some components of the MLPs' inputs (those that come from model components not in the circuit) receive input from the 01-dataset as well. We stress that this is a difficult task, where the large majority of the model receives input that should push it to poor performance. For a diagram of the circuit and our evaluation, see Figure 5.

We perform this evaluation using the larger dataset, and almost entirely recover model performance. The probability difference is 72.7% (89.5% of the original) and the cutoff sharpness is 8%—sharper than pre-patching. This indicates that our circuit is mostly sufficient to compute this task. The circuit is also necessary: performing the opposite of the prior experiment, giving nodes in our circuit the 01-dataset, and those outside it the normal dataset, leaves GPT-2 unable to perform the task: it achieves a probability difference of -36.6%.

If we target other circuits of a size and location similar to our circuit's, performance is related to the preservation of the paths from the input, to our attention heads, our MLPs (especially MLPs 9 and 10), to the logits. If the paths are interrupted (i.e. no attention head or no MLP overlap with the original circuit), performance is very low. If at least one path is preserved, performance improves with each additional component in common with the original circuit; MLPs have the biggest impact.

3.3 Circuit Semantics

Now, we interpret each circuit component, starting with the attention heads. We first perform a simple attention-pattern analysis of the heads in our circuit. Figure 6 shows which tokens our attention heads attend to at which positions. At the relevant (end) position, in-circuit attention heads attend to the YY position, suggesting that they detect the year which the output year must be greater than.

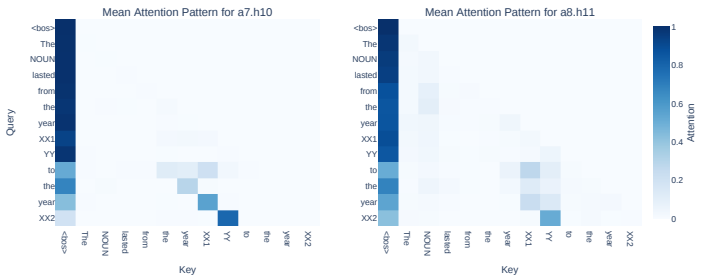

Figure 6: Attention patterns of a7.h11 and a8.h10. <bos> denotes GPT-2's start of sentence token.